HXC-031 Plan

HXC-031 — Implementation Plan: Codex Multimodal + Cline Computer Use Port

Field	Value
Revision	1
Created	2026-05-29
Last modified	2026-05-29
Status	DRAFT — scoping/plan only, NO code (operator decision pending)
Tracker item	HXC-031 (Type: Feature)
Author	Research subagent (read-only analysis)
Authority	§11.4.70 subagent-driven, §11.4.74 catalogue-first, §11.4.50/§11.4.98 anti-bluff test plan

---

Table of Contents

---

1. Scope & Method

Read-only analysis of the in-repo reference sources and the HelixCode integration surface. No source modified; this document is the sole output. All capability characterisations below cite the exact files read (§11.4.6 no-guessing).

Files read for characterisation: - `cli_agents/codex/codex-rs/protocol/src/models.rs` (lines 695–723, 1053–1090) — Codex content-item model +

local-image → data-URL construction. - `cli_agents/codex/codex-rs/protocol/src/items.rs` (lines 289–303) — `image_urls()` / `local_image_paths()` / `UserInput::LocalImage{path}`. - `cli_agents/codex/codex-rs/protocol/src/dynamic_tools.rs` (line 47) — `InputImage { image_url }`. - `cli_agents/cline/src/core/prompts/system-prompt/tools/browser_action.ts` (whole file) — the `browser_action` tool spec (launch/click/type/scroll/close, coordinate-based, screenshot-per-action). - `cli_agents/cline/src/services/browser/BrowserSession.ts` (lines 1–90) — Puppeteer-driven session, screenshot capture. - `helix_code/internal/llm/missing_types.go` (lines 122–165) — `Message{Role,Content string}` (TEXT-ONLY) + `LLMRequest`. - `helix_code/internal/llm/anthropic_provider.go` (lines 61–90, 326, 430–520, `convertMessages`) — wire-level `anthropicContentBlock`+`anthropicImageSource` exist, but `convertMessages` is text-only. - `helix_code/internal/llm/vision/{doc.go,*}` — existing vision detection + model-switch package. - `helix_code/internal/tools/browser/` + `helix_code/internal/tools/browser_*_v2.go` — the P2-F23 chromedp browser suite. - `helix_code/internal/tools/browser_screenshot_v2.go` — screenshot tool returns an **absolute file path, NOT base64**. - `dependencies/vasic-digital/vision_engine/` (`pkg/llmvision/`, `pkg/analyzer/`) — owned VisionEngine submodule, already wired in `.gitmodules` + `helix_code/go.mod` replace `digital.vasic.visionengine`. - `docs/CONTINUATION.md` — Phase-2 port precedent (P2-F23 Cline Browser Tool CLOSED, P2-F29 Roo-code CLOSED).

---

2. Capability Summary

2.1 Codex Multimodal

What it is (evidenced): Codex ingests images (screenshots, pasted images, file paths) and routes them to the model as structured content. The model (`protocol/src/models.rs`) is a content-item union:

```
pub enum ContentItem {
    InputText  { text: String },
    InputImage { image_url: String, detail:
        Option<ImageDetail> }, // image_url is a data: URL
    OutputText { text: String },
}
pub enum ImageDetail { Auto, Low, High,
        Original }   // DEFAULT = High
```

A user-supplied local image (`UserInput::LocalImage { path }`, `items.rs:303`) is read, validated (MIME, decode), and converted to a base64 **data URL** via `local_image_content_items_with_label_number` (`models.rs:1053`), then pushed as `ContentItem::InputImage { image_url: image.into_data_url(), detail: High }`, optionally bracketed by `InputText` label tags. Unsupported/oversized/corrupt images become a placeholder `InputText` error item (graceful degradation — never a crash). A turn's content is `Vec<ContentItem>` (mixed text + images).

So “Codex Multimodal” = a per-message list of content blocks where each block is text OR an image, the image carried as a base64 data URL with a detail hint, with robust local-file → data-URL ingestion and graceful error placeholders.

“Done” for HelixCode users: a HelixCode user can attach an image (file path, `data:` URI, or a tool-produced screenshot) to a prompt; that image actually reaches a vision-capable provider (Anthropic/OpenAI/Gemini/Ollama-vision) as an image content block; the model’s response demonstrably depends on the image content (captured runtime evidence — not “image was attached” metadata).

2.2 Cline Computer Use

What it is (evidenced): Cline’s “computer use” is its `browser_action` tool (`browser_action.ts`) — NOT OS-level mouse/keyboard control. It drives a **Puppeteer-controlled browser** (`BrowserSession.ts`, `puppeteer-core` + `chrome-launcher`) with a fixed action set:

- `launch <url>` (must be first), `click <x,y>`, `type <text>`, `scroll_down`, `scroll_up`, `close` (must be last).
- Viewport is a fixed resolution; clicks are **coordinate-based**, derived by the model from a **screenshot**.
- **Every action (except `close`) returns a screenshot of the new browser state + new console logs**, which is fed back to the model to decide the next action. Only `browser_action` may be used while the browser is open.

So the loop is: action → screenshot → (image fed to model) → next action. The “computer use” intelligence lives entirely in that **screenshot-feedback loop**; the actuation is browser automation, not raw screen control.

“Done” for HelixCode users: a HelixCode user (or the agent autonomously) can issue a high-level UI goal (“open localhost:3000 and click the Login button”); HelixCode drives a real headless browser, captures a screenshot after each step, feeds that screenshot back to a vision model, and the model issues the next coordinate-based action — closing the loop end-to-end with captured visual evidence at each step.

---

3. HelixCode Integration Surface

Concern	Status in HelixCode	Evidence
Browser automation (launch/click/type/scroll/screenshot/close)	**ALREADY EXISTS** — P2-F23 chromedp suite	`internal/tools/browser/`, `browser_{navigate,click_type,scre` `_v2.go`, `browser_register_v2.go`
Coordinate-based click + type	EXISTS (`browser_click_type_v2.go`)	tools dir listing

Concern	Status in HelixCode	Evidence
Screenshot capture	EXISTS but returns **file path, not base64**	`browser_screenshot_v2.go:30` “Retu (NOT base64) per spec §3.4”
Vision detection + auto model-switch	EXISTS	`internal/llm/vision/` (detector, regi
Wire-level image content block (Anthropic)	EXISTS in serializer	`anthropic_provider.go:66-90` `anthr` `anthropicImageSource{Type:base64`
Request-level image carriage (`llm.Message`)	**MISSING** — `Message{Role,Content string}` is text-only	`missing_types.go:122`
`convertMessages` image mapping	**MISSING** — drops everything but `msg.Content` string	`anthropic_provider.go` `convertMess`
Multimodal carriage for OpenAI/Gemini/ Ollama/Bedrock providers	**MISSING** at request level	provider files map `Content` string only
CLI image/attach input	partial: text `<attached_files>` block only (file contents inlined as text)	`cmd/cli/main.go:1694-1730`
Agent tool dispatch	EXISTS (`agent/tool_dispatch.go`) — where a new tool/loop plugs in	agent dir listing
VisionEngine owned submodule (LLM-vision + screen analyzer)	**WIRED** (submodule + go.mod replace) but **not consumed** by helix_code	`.gitmodules:319-321`, `helix_code/g`

Conclusion: The dominant gap is a **request-level multimodal pipe**: images can be *detected* (`vision/`) and *serialized at the wire* (anthropic block), but the canonical `llm.Message.Content string` has no slot to carry an image, so an attached image cannot travel from CLI/agent → provider. Cline-computer-use actuation is essentially done; what is missing for a real autonomous loop is the **screenshot-as-image-feedback** — which itself depends on the multimodal pipe and on the screenshot tool being able to hand back image bytes (currently path-only).

---

4. Catalogue-Check Result (§11.4.74)

Capability	Verdict	Detail
Cline Computer Use (browser actuation)	**reuse (in-repo)** — `helix_code/internal/tools/browser` @ P2-F23	The 6-tool chromedp suite already exists. No new browser engine. Extend only:

Capability	Verdict	Detail
		screenshot bytes + feedback loop.
Codex Multimodal (vision/LLM-vision)	**extend `vasic-digital/VisionEngine`** (`dependencies/vasic-digital/vision_engine`, module `digital.vasic.visionengine`)	`pkg/llmvision/` already has anthropic/openai/gemini/ollama/qwen/kimi/astica vision providers + `pkg/analyzer/` (AnalyzeScreen, CompareScreens, ScreenDiff). It is **owned, wired in go.mod**, but **not yet consumed**. Reuse its image-encode + LLM-vision providers; extend with any missing pieces upstream (PR to VisionEngine), never duplicate in-project (CONST-051(B)/§11.4.74).
Screen capture / UI recording	**reuse `vasic-digital/Panoptic`** (already submodule, `.gitmodules:225`) if a record-the-session evidence artefact is wanted for the computer-use Challenge	optional, evidence-layer only
Visual diffing of screenshots	**reuse `vasic-digital/ScreenDiff`** if step-to-step visual assertion is desired	optional

Net catalogue verdict: REUSE + EXTEND, do NOT build fresh. Browser actuation = reuse in-repo P2-F23. Vision/LLM-vision = extend the owned VisionEngine submodule. The only genuinely new HelixCode code is the **request-level multimodal carriage** (the `llm.Message` content-block change + per-provider mapping) and the **wiring** that connects screenshot → VisionEngine/provider → agent loop.

---

5. Architecture / Design

5.1 Capability A — Multimodal LLM request surface

Design goal: add an image-bearing content-block representation to the canonical request type and map it through every provider, mirroring Codex's `ContentItem` union. Keep `Message.Content string` working (back-compat) by adding an OPTIONAL parallel field rather than changing its type.

New/changed types (`helix_code/internal/llm/`): - MODIFY `missing_types.go`: - Add `type ContentPart struct { Type string; Text string; ImageURL string; Detail string }` (Type ∈ `text|image`), mirroring Codex `ContentItem` (data-URL in `ImageURL`, `Detail` ∈ auto/low/high). - Add `Parts []ContentPart`json:“parts,omitempty”``to`Message`(optional; when empty, fall back to`Content string`– zero behaviour change for text-only callers). - NEW`helix_code/internal/llm/multimodal.go`(~200 lines):`func BuildImagePart(pathOrURI string, detail string) (ContentPart, error)`– Codex-equivalent local-file→data-URL: read bytes, sniff MIME (magic numbers, reuse`vision/detector.go`), base64-encode to`data:;base64,…`, enforce a max-size + supported-format set, return a graceful text placeholder part on unsupported/oversized (NOT an error to the turn) — matching`models.rs`placeholder behaviour. Prefer delegating the encode/MIME logic to`digital.vasic.visionengine` if it exposes it (extend VisionEngine to export it if not).

Per-provider mapping (MODIFY, each its own ≤300-line task): - `anthropic_provider.go` `convertMessages`: when `msg.Parts` non-empty, emit `[]anthropicContentBlock` with `Type:"image"`, `Source:{Type:"base64",MediaType,Data}` (the wire structs already exist — just stop dropping them). - `openai_compatible_provider*.go` / openai-family: map to OpenAI `content: [{type:"image_url", image_url:{url, detail}}]` (Codex’s exact shape). - `gemini_provider.go`: map to `inlineData{mimeType,data}` parts. - `bedrock_provider.go` (Anthropic-on-Bedrock): same as anthropic block. - `ollama_provider`/`llamacpp_provider`: map to `images: [base64]` array. - Providers whose configured model is NOT vision-capable: route through existing `internal/llm/vision/switcher.go` to auto-switch (or surface a CONST-046 i18n message), reusing the EXISTING vision package.

CLI surface (MODIFY `helix_code/cmd/cli/main.go`): - Add `-image` <`path|uri`> (repeatable) flag → builds `ContentPart` image parts on the user `Message` (distinct from the existing text-only <`attached_files`> path).

Data flow (A):

```
CLI -image / agent screenshot
   → BuildImagePart() [VisionEngine encode] → ContentPart{image, data-URL}
   → llm.Message.Parts
   → provider.convertMessages() maps to provider-native image block
   → real HTTP call to vision model
   → response depends on image content (captured evidence)
```

5.2 Capability B — Computer-use feedback loop

Design goal: close Cline’s screenshot→model→action loop on top of the already-ported browser suite, using Capability A as the transport for the screenshot image.

New/changed (mostly wiring, browser engine is reused): - MODIFY `browser_screenshot_v2.go` (or add a sibling `browser_screenshot_bytes`): return base64/`data:` bytes in addition to the file path, so the agent can feed the screenshot back to the model as a `ContentPart` image. (Today it is path-only — this is the load-bearing gap for the loop.) - NEW `helix_code/internal/agent/computer_use.go` (~250 lines): a loop driver that, given a high-level goal, iterates: send model turn → parse a `browser_action`-style tool call (reuse existing v2 tools: navigate/click_type/scroll/screenshot/close) → execute via the existing browser tool registry → capture screenshot bytes → append as `ContentPart` image to the next turn → repeat until model emits a completion or a step budget is hit. Mirrors Cline's "only browser_action while open; must launch first, close last" discipline. - NEW `/computer-use` (or `/browse-goal`) slash command in the CLI, paralleling the existing `/browser` slash from P2-F23. - OPTIONAL evidence: wrap the session with `vasic-digital/Panoptic` recording and/or `ScreenDiff` per-step visual assertions (evidence layer only; reuse, not build).

Data flow (B):

```
goal → agent loop turn (vision model)
   → model returns coordinate action (click 450,300 / type /
scroll)
   → existing browser_*_v2 tool executes via chromedp
   → browser_screenshot returns bytes → BuildImagePart (Cap. A)
   → screenshot ContentPart appended → next turn
   → ... until model 'close' + attempt_completion
```

Why B mostly depends on A: without the multimodal request surface, the screenshot cannot be fed back to the model and the loop degenerates to blind coordinate guessing. Therefore **A must land before B**.

---

6. Build Sequence

Subagent-friendly, each unit ≤~300 lines per §11.4.70/§11.4.82(G). Worktree isolation default. Tasks T01–T05 are Capability A (must precede B).

Task	Title	Files	Est. LoC
T01	`ContentPart` + `Message.Parts` types + unit tests	`internal/llm/missing_types.go`, `_test.go`	~120
T02	`multimodal.go` local-file/URI → data-URL encoder (delegate to VisionEngine encode; extend VisionEngine upstream if it lacks an exported encoder) + graceful placeholder + unit tests	`internal/llm/multimodal.go`, `dependencies/vasic-digital/vision_engine/pkg/llmvision/*` (extend)	~250
T03		`anthropic_provider.go`, `bedrock_provider.go`	~200

Task	Title	Files	Est. LoC
	Anthropic + Bedrock `convertMessages` image mapping + tests		
T04	OpenAI-compatible + Gemini + Ollama/llamacpp image mapping + tests	`openai_compatible_provider*.go`, `gemini_provider.go`, `ollama/llamacpp`	~280
T05	CLI `-image` flag + non-vision-model auto-switch wiring (reuse `vision/switcher.go`) + tests	`cmd/cli/main.go`	~150
T06	`browser_screenshot` returns image bytes/data-URL (sibling tool or option) + tests	`internal/tools/browser_screenshot_v2.go`, `browser/screenshot.go`	~150
T07	`computer_use.go` agent loop driver (action→screenshot→feedback) + unit tests	`internal/agent/computer_use.go`	~280
T08	`/computer-use` CLI slash command + dispatch wiring	`cmd/cli/main.go`, agent dispatch	~120
T09	Integration tests (real provider + real chromedp) — see §7	`tests/integration/`	~250
T10	E2E + HelixQA bank/ Challenge + docs + CONTINUATION/tracker close-out	`tests/e2e/`, HelixQA bank, `docs/`	~250

Sequencing: T01→T02→{T03,T04 parallel}→T05 (Cap. A complete) → T06→T07→T08 (Cap. B) → T09→T10. T03/T04 are disjoint-file and PWU-parallelisable (§11.4.58).

---

7. Test + Challenge Plan

All non-unit tests use real infrastructure (CONST-050(A)/Rule 5) and must be fully self-driving with no human-in-loop (§11.4.98). Each PASS carries captured positive evidence (§11.4.5/§11.4.69) — never config/metadata-only.

Capability A — Multimodal

- **Unit** (mocks OK here only): `ContentPart` round-trip; `BuildImagePart` on a fixture PNG → asserts a valid `data:image/png;base64,` URL with correct MIME; oversized/corrupt fixture → asserts graceful text placeholder (not error); per-provider `convertMessages` emits the provider-native image block.
- **Integration** (real HTTP, real vision model): construct a turn with a **known, distinctive fixture image** (e.g. a PNG containing the text

"HELIX-4F2A" or a red square in a known quadrant). Call a real vision provider. **Anti-bluff evidence shape:** the response text MUST contain the unforgeable token rendered in the image (e.g. asserts response contains `4F2A`) — proving the model actually saw the image, not the filename/prompt. Capture request+response JSON under `docs/qa/HXC-031/<run-id>/`. Re-runnable: `-count=3` consecutive PASS.
- **E2E**: `bin/cli generate --model <vision> --image fixtures/helix-token.png "What 4-char code is in this image?"` → stdout contains the token. Captured terminal transcript.

Capability B — Computer-use loop

- **Integration** (real chromedp + real vision model): serve a local fixture page with a uniquely-labelled button at a known coordinate; give the agent the goal "click the GREEN button". **Sink-side evidence (§11.4.69, taxonomy `gpu_render`/UI-action proxy):** the fixture page's button onclick writes a sentinel to the DOM / a local file; the test asserts that sentinel is present AFTER the loop — proving the click physically landed where the screenshot-driven model aimed. Plus per-step screenshot artefacts saved under the run-id dir. Re-runnable ×3.
- **E2E / HelixQA bank**: a new self-driving bank `helixqa/banks/computer_use/` driving `bin/cli` via `os/exec` against the local fixture server + headless chromium, asserting the DOM sentinel + capturing the screenshot sequence. No manual review required (§11.4.98). Pair with a flip-mutation (break the coordinate mapping → bank must FAIL).
- **Stress/Chaos (§11.4.85)**: ≥10 concurrent browser sessions (resource contention, no FD leak / no orphan chromium); chaos — kill the chromium PID mid-loop → driver must degrade cleanly (close + error), never crash.

Self-driving / operator-attended flag (§11.4.98)

- **Capability A is fully self-driving** (fixtures + real provider, no human).
- **Capability B is self-driving for the BROWSER variant** (headless chromium + fixture server) — this is exactly why Cline's "computer use" is browser-scoped, and why the in-repo P2-F23 chromedp port is the right substrate. **It is NOT real OS-level screen/mouse control.** If the operator later wants true OS-level computer-use (X11/Wayland/`xdotool`/CGEvent), that is **inherently operator-attended / sandbox-required** (a real desktop session) — flag it as out-of-scope here and a separate design fork (see §8). Sink-side evidence for any OS-level variant would be the §11.4.69 `gpu_render`/`touch_input` taxonomy (screencap diff + the actuated app's own state change), and the test would be SKIP-with-reason `operator_attended/hardware_not_present` on headless CI rather than a faked PASS.

---

8. Effort Estimate, Risks, Design Forks

Effort estimate (engineering, excluding review/QA latency): - Capability A (T01–T05): ~3–4 focused subagent sessions. The wire structs already exist for Anthropic; the work is type-plumbing + per-provider mapping + the encoder (mostly reused from VisionEngine). - Capability B (T06–T08): ~2–3 sessions; browser engine reused, so this is loop + feedback wiring. - Tests + Challenge + docs (T09–T10): ~2 sessions. - **Total: ~7–9 subagent sessions / roughly 1.5–2 engineering-days of focused work**, assuming VisionEngine already exposes (or is cheaply extended to expose) an image encoder. Lower than a from-scratch port because actuation + vision detection + wire serialization already exist.

Risks: 1. **Provider image-format divergence** — each provider wants a different JSON shape (Anthropic base64 source vs OpenAI image_url vs Gemini inlineData vs Ollama images-array). Mitigation: one mapping function per provider, table-tested against fixtures; do NOT leak a single provider's shape into the canonical type (keep `ContentPart` provider-neutral, Codex-style). 2. **CONST-036/037 model capability —** only vision-capable models may receive images; routing a non-vision model an image is a hard error. Mitigation: reuse the existing `vision/switcher.go` + verifier capability flags (CONST-040), never hardcode a vision-model list. 3. **VisionEngine coupling (CONST-051(B))** — must extend VisionEngine in a project-unaware way (no HelixCode-specific context injected); consume via its public `pkg/llmvision/pkg/analyzer` API. Any missing encoder → PR upstream, bump pointer. 4. **Screenshot size / token cost** — full-page screenshots can be large; enforce viewport-size + max-bytes + `detail:low` default for loop steps (Codex uses `detail:High` default for user images but a loop should economise). 5. **Chromium availability in CI/headless** — the P2-F23 suite already solved discovery; reuse `browser/discovery.go`. Integration tests gate on real chromium presence (honest SKIP if absent, never faked PASS). 6. **CONST-046 (no hardcoded content)** — every new user-facing string (CLI flag help, loop status, errors) must be i18n via the existing `tools/i18n/trc(...)` pattern already used in `cmd/cli/main.go` and the browser tools. No raw English literals.

Genuine design forks the operator must decide: - **FORK-1 (scope of "computer use"): browser-only vs real OS-level screen control.** The evidenced Cline capability is **browser-only** (Puppeteer), and HelixCode already has that substrate. Recommendation: ship browser-scoped computer-use (self-driving, anti-bluff-clean). Real OS-level screen/mouse control is a separate, inherently operator-attended, sandbox-and-safety-heavy effort — decide whether it is even in HXC-031's scope. **This is the biggest fork.** - **FORK-2 (which multimodal models / providers first).** Anthropic Claude is the cheapest first target (wire structs already exist). Decide the launch provider set (Claude only vs Claude+OpenAI+Gemini+Ollama-vision at once). Affects T04 size. - **FORK-3 (VisionEngine reuse depth).** Either (a) consume VisionEngine's `pkg/llmvision` providers directly as the vision path (heavier integration, more reuse, possibly duplicates the existing `internal/llm/vision` switching), or (b) use only VisionEngine's image-encode/analyzer helpers and keep image transport in `internal/llm` providers (lighter, recommended). Operator should pick (b) unless they want to consolidate all vision onto VisionEngine.

---

9. §11.4.99 Sources note

This plan's capability characterisations are derived **entirely from the in-repo reference source** (the `cli_agents/codex/` and `cli_agents/cline/` checkouts) and the HelixCode tree as listed in §1 — NOT from external/online documentation or training memory. No operator-facing setup instructions for the live Codex/Cline products are emitted here, so the §11.4.99 latest-online-docs cross-reference obligation does not bind this scoping document. **If implementation (HXC-031 build phase) adds any operator-facing guide that cites live OpenAI Codex multimodal API shapes or Anthropic/Gemini/OpenAI vision request schemas, that guide MUST fetch and cross-reference the latest official provider docs and carry a `## Sources verified <date>: <urls>` footer per §11.4.99 before commit.**

Sources verified

- In-repo reference source only (no external fetch): `cli_agents/codex/codex-rs/protocol/src/{models.rs,items.rs,dynamic_tools.rs}`, `cli_agents/cline/src/core/prompts/system-prompt/tools/browser_action.ts`, `cli_agents/cline/src/services/browser/BrowserSession.ts`, HelixCode `internal/llm/*`, `internal/tools/browser*`, `dependencies/vasic-digital/vision_engine/*`, `docs/CONTINUATION.md`. Date: 2026-05-29.